

Resumo Executivo

Este documento detalha a **evolução das arquiteturas de integração de dados**, explorando a transição do modelo **ETL** tradicional para o paradigma **ELT** moderno e a **Arquitetura Medallion**. Analisamos ferramentas históricas do ecossistema **Hadoop**, suítes corporativas tradicionais, plataformas focadas em **DataOps** e as novas bibliotecas programáticas. Por fim, abordamos a integração com **formatos de tabela abertos** (Iceberg, Delta Lake e Hudi) e tecnologias de baixa latência como **CDC**.

O Ecossistema de Ingestão de Dados em Data Lakes: Evolução Histórica, Plataformas de Integração e Paradigmas Modernos

A Transição Macro-Arquitetural: Da Origem do ETL ao Paradigma ELT e o Modelo Medallion

O panorama da engenharia de dados passou por transformações estruturais profundas desde o seu surgimento na década de 1970.[1] Naquela era pioneira, as organizações que lidavam com dados dispersos precisavam consolidar suas informações de forma manual, evoluindo para processos automatizados no final da década de 1980.[1] Esse cenário deu origem ao paradigma tradicional conhecido como ETL (*Extract, Transform, Load*).[2, 3] O ETL consolidou-se em uma época marcada por recursos computacionais escassos, armazenamento físico on-premises de alto custo e acoplamento rígido entre computação e armazenamento.[1, 2] Sob essa ótica, as transformações precisavam ocorrer em uma camada de processamento intermediária e dedicada antes que o dado purificado fosse persistido no destino final, uma vez que os bancos de dados e *data warehouses* de legado eram lentos e limitados demais para suportar cargas de computação pesadas.[1, 2]

A arquitetura ETL clássica oferece vantagens expressivas para casos de uso específicos.[1, 4] Ao processar e limpar as informações antes do carregamento, os pipelines garantem um nível elevado de qualidade e governança corporativa.[1, 3] Esse modelo facilita o mascaramento de dados sensíveis, a filtragem de informações de identificação pessoal (PII) e a aplicação de controles rígidos de conformidade regulatória antes que os registros toquem o armazenamento definitivo.[1, 3, 4] Contudo, o descarte sistemático de dados brutos imposto pelo ETL atua como um limitador crítico para a ciência de dados moderna, pois impossibilita reanalisar o histórico de transações sob novas regras de negócios sem reextrair todas as fontes primárias.[1, 4]

A transição para o paradigma ELT (*Extract, Load, Transform*) consolidou-se na década de 2010 com a ascensão da computação em nuvem e a consequente separação elástica de computação e armazenamento.[1, 2] No ELT, os dados brutos são extraídos e imediatamente carregados em sua forma original no destino final, delegando a

responsabilidade pelas transformações ao motor computacional distribuído do próprio repositório de destino.[2, 3] Esse processo baseia-se no conceito de esquema na leitura (*schema-on-read*), garantindo que dados estruturados, semiestruturados ou não estruturados coexistam de forma econômica no ambiente analítico.[1, 3] A velocidade e a flexibilidade desse modelo fundamentam-se na capacidade de processamento paralelo massivo (MPP) das modernas tecnologias de armazenamento analítico em nuvem.[1] No entanto, convém pontuar que a vantagem de velocidade do ELT pode desaparecer sob certas condições limítrofes, como o uso de dimensões de computação pequenas incapazes de paralelizar tarefas, ambientes com alta concorrência de consultas sem isolamento de carga de trabalho, processos dispendiosos de reproprocessamento histórico (*backfills*) que realizam varreduras completas em tabelas massivas ou transformações de alta complexidade que não se beneficiam de armazenamento colunar.[1]

Para organizar esse fluxo de dados brutos e garantir que o ecossistema analítico permaneça sustentável, a engenharia de dados moderna consolidou a Arquitetura Medallion.[4] Esse padrão organiza as camadas de armazenamento no data lake ou data lakehouse em três zonas físicas e lógicas distintas:

Camada Medallion	Estado do Dado	Objetivo Técnico e Governança
Bronze (Raw)	Bruto / Original	Atua como a fonte única da verdade histórica. Os dados são carregados exatamente como extraídos dos sistemas de origem, sem filtrações ou conversões, permitindo reproprocessamentos retroativos ilimitados.[4]
Silver (Cleaned)	Validado / Higienizado	Os dados brutos passam por processos de validação de esquemas, padronização de tipos de dados, eliminação de registros duplicados e enriquecimento inicial. Esta camada é comumente modelada e gerenciada via ferramentas de engenharia de software como dbt.[4]
Gold (Business-Ready)	Agregado / Consolidado	Camada otimizada para consumo final, onde regras de negócios complexas são aplicadas e as tabelas são estruturadas em modelos estrela ou agregados prontos para alimentar painéis de Business Intelligence (BI) e algoritmos de Machine Learning.[4]

À medida que essas arquiteturas amadureceram, surgiu a necessidade de alimentar os sistemas operacionais de negócios com as inteligências derivadas da camada analítica Gold.[2] Esse movimento inverso deu origem ao padrão de *Reverse ETL*, o qual transfere dados transformados e enriquecidos do data warehouse de volta para ferramentas de CRM,

ERP e plataformas de marketing SaaS, automatizando ações de negócios diretamente nos pontos de contato operacionais.[2]

A Era Pioneira do Ecossistema Hadoop: Apache Sqoop e Apache Flume

Com o surgimento e a popularização do Apache Hadoop no início dos anos 2000, o mercado de tecnologia deparou-se com a necessidade premente de ingerir petabytes de dados vindos de fontes legadas em sistemas de arquivos distribuídos (HDFS).[5] Nesse cenário, o ecossistema Hadoop introduziu duas ferramentas que definiram os primeiros padrões de ingestão de dados em larga escala: o Apache Sqoop e o Apache Flume.[6]

Apache Sqoop

O Apache Sqoop foi desenvolvido para atuar como uma interface de linha de comando especializada no transporte bidirecional de dados em lote entre bancos de dados relacionais (RDBMS), mainframes (como DB2) e o sistema HDFS ou tabelas do Apache Hive.[6, 7] O mecanismo interno do Sqoop baseia-se diretamente no modelo computacional MapReduce.[6] Ao disparar um comando de importação, o Sqoop conecta-se à origem via JDBC para ler os metadados da tabela.[7] Em seguida, o algoritmo calcula os limites de partição lógica dos dados com base na chave primária da tabela ou em uma coluna específica configurada pelo argumento `split-by`. [7] Com base nesses limites, o Sqoop gera de forma automatizada um código-fonte Java estruturado, compila-o em um arquivo JAR e submete um job MapReduce ao cluster.[7]

Uma característica arquitetural singular do Sqoop é que ele executa seus jobs utilizando exclusivamente tarefas do tipo *Mapper*. Once que o Sqoop executa apenas transferências diretas de dados sem realizar agregações, agrupamentos ou ordenações complexas em tempo de ingestão, a etapa de *Reducer* é omitida, otimizando o consumo de rede e disco do cluster Hadoop.[7] O Sqoop também dispõe de suporte para cargas incrementais em dois modos operacionais distintos: o modo de `append` (adequado para tabelas que recebem apenas novas inserções de registros de forma sequencial) e o modo de `data de última modificação` (projetado para capturar inserções e atualizações combinadas).[7]

Apesar de sua ampla adoção histórica, o Sqoop enfrentava problemas graves de evolução tecnológica. À medida que o mercado migrava do Hadoop 2.0 para distribuições modernas como o Hadoop 3.3.1, a rigidez do código base do Sqoop causava erros frequentes de implementação de APIs depreciadas, gerando incompatibilidades críticas em ambientes modernos.[8] Diante da diminuição contínua de contribuições da comunidade, da ausência de novos committers e da escassez de lançamentos de segurança estáveis (com a versão 1.4.7 mantendo-se inalterada por anos), o comitê de gerenciamento do projeto recomendou sua descontinuação, culminando na transferência oficial do Apache Sqoop para o Apache Attic em junho de 2021.[9, 10]

Apache Flume

Diferente da natureza orientada a lotes do Sqoop, o Apache Flume foi projetado pela Cloudera para lidar com a coleta, agregação e movimentação de fluxos contínuos de arquivos de log e dados de eventos em tempo de execução quase real.[6, 11, 12] O Flume baseia-se em uma arquitetura de fluxo distribuída composta por agentes Java independentes operando como daemons de sistema.[11, 13] Cada agente Flume abriga três componentes integrados de forma transacional [6, 14]:

1. **Source:** Responsável por escutar e receber os dados de fontes geradoras externas (como servidores Web ou a API de streaming do Twitter) e encapsular esses registros em blocos de dados conhecidos como *Flume Events*. [11, 12, 14]
2. **Channel:** Funciona como uma fila ou repositório de trânsito temporário que retém os *Flume Events* localmente até que o destino esteja pronto para gravá-los. [6, 11, 13] O canal pode ser baseado em memória RAM (priorizando taxa de transferência e latência ultrabaixa) ou em disco rígido (garantindo persistência física e recuperação em caso de travamento do agente). [12, 14]
3. **Sink:** Extrai os eventos armazenados no Channel e os grava de forma definitiva no destino de armazenamento final (como HDFS, Apache HBase ou índices do Elasticsearch através do conector nativo *ElasticsearchSink*). [6, 12, 13]

O design do Flume oferece flexibilidade para a construção de topologias de rede complexas. [13] Ele permite encadear múltiplos agentes em arquiteturas multi-hop, além de suportar topologias de consolidação (fan-in, onde múltiplos agentes de borda enviam dados para um único agente concentrador) e de distribuição (fan-out, onde um único agente replica o mesmo evento para múltiplos destinos com roteamento contextual). [13] Para garantir a confiabilidade de transmissão física de dados de ponta a ponta, o Flume apoia-se em um modelo transacional de duas fases para a transferência de eventos entre saltos, além de disponibilizar modos de criptografia de canal e autenticação de agentes via SASL (*Simple Authentication and Security Layer*). [11, 14]

Entretanto, as limitações do Flume tornaram-se evidentes em ambientes de produção de alta escala. [12, 14] O framework fornece garantias de entrega do tipo *at-least-once*, mas apresenta uma ausência completa de ordenação sequencial dos eventos consumidos. [14] O dimensionamento de hardware para canais baseados em disco rígido provou-se altamente complexo e sujeito a gargalos severos de leitura e gravação física. [14] Com a consolidação do Apache Kafka no mercado — que atua como uma plataforma de streaming distribuída de altíssimo desempenho, tolerância a falhas superior e que funciona como um superconjunto das capacidades de ingestão do Flume —, a prioridade de desenvolvimento da Cloudera e do mercado migrou, levando o Apache Flume ao declínio técnico e à obsolescência prática. [6, 13]

Dimensão Técnica	Apache Sqoop	Apache Flume
------------------	--------------	--------------

Padrão de Ingestão	Lote (<i>Batch</i>) e Bidirecional [6]	Fluxo Contínuo (<i>Streaming</i>) e Unidirecional [6]
Fontes de Origem	Bancos relacionais (RDBMS), NoSQL e mainframes [6]	Arquivos de log de servidores web, syslogs e redes sociais [6, 11]
Mecanismo Interno	Geração e execução de mappers MapReduce [6, 7]	Agentes Java autônomos (Source, Channel e Sink) [6, 13]
Garantia de Entrega	Consistência transacional JDBC baseada na tabela de origem	Entrega do tipo <i>at-least-once</i> sem garantia de ordenação estrutural [14]
Status Atual	Descontinuado [9]	Obsoleto [6]

Ferramentas de Integração Corporativa Tradicional: PowerCenter, ODI e Talend

Antes da popularização dos ambientes elásticos de computação em nuvem, as grandes corporações estruturavam seus sistemas de integração de dados em torno de suítes de middleware proprietárias e centralizadas.[15] Essas ferramentas de nível empresarial atendiam a requisitos complexos de governança e modelagem de dados on-premises, porém apresentavam altos custos de manutenção de infraestrutura física e ciclos de desenvolvimento lentos.[15, 16]

Informatica PowerCenter

Fundado em 1993, o Informatica PowerCenter consolidou-se como o líder de mercado para processamento ETL tradicional em ambientes corporativos de grande porte.[15] Sua arquitetura de execução apoia-se em um motor de processamento proprietário altamente otimizado para realizar operações de transformação de dados em memória, atuando em um servidor intermediário dedicado.[17, 18] No PowerCenter, os fluxos lógicos de dados são projetados em componentes visuais chamados *Mappings*, enquanto a orquestração física e as parametrizações operacionais de conexões com bancos de dados, arquivos e tabelas são configuradas em blocos separados conhecidos como *Sessions*. [18]

O principal desafio associado ao PowerCenter reside na sua rigidez operacional e no acoplamento com hardware dedicado on-premises.[15, 17] Para escalar horizontalmente os processos de transformação corporativos à medida que o volume de dados cresce, as organizações enfrentam custos de licenciamento proibitivos e alta complexidade de administração de sistema.[17] Na tentativa de modernizar seu portfólio para a nuvem pública, a Informatica lançou o Informatica Intelligent Data Management Cloud (IDMC) como o sucessor nativo em nuvem do PowerCenter.[16] O IDMC adota um modelo de cobrança baseado em consumo mensurado por Unidades de Processamento Informatica (IPUs), que, embora reduza a necessidade de provisionamento local de hardware, apresenta menor

previsibilidade de custos operacionais e pode desincentivar as equipes de engenharia de processar volumes crescentes de dados devido ao risco de estouro orçamentário.[16] Adicionalmente, a transição para o ecossistema IDMC (atualmente integrado de forma profunda à nuvem fechada da Salesforce) gerou preocupações no mercado sobre a perda de flexibilidade multicloud e o risco de aprisionamento tecnológico (vendedor lock-in).[19]

Oracle Data Integrator (ODI)

Em direta oposição à arquitetura baseada em servidores de processamento dedicados da Informatica, o Oracle Data Integrator (ODI) adota um paradigma de design ELT puramente declarativo.[20] O ODI elimina a necessidade de um motor de transformação intermediário proprietário ao utilizar a própria capacidade de processamento dos bancos de dados relacionais e analíticos de destino (estratégia de processamento *push-down* em banco).[20, 21] No fluxo de trabalho do ODI, os dados são extraídos das fontes, carregados diretamente em tabelas temporárias ou de transição (*staging*) na base de dados de destino e transformados por meio de consultas SQL nativas geradas automaticamente pela plataforma.[20, 22, 23]

A modularidade e extensibilidade do ODI sustentam-se no conceito de Módulos de Conhecimento (Knowledge Modules - KMs).[22] Os KMs são modelos de código reutilizáveis e parametrizáveis que implementam estratégias de transformação e movimentação física otimizadas para tecnologias específicas.[22, 24] A plataforma fornece seis tipos fundamentais de KMs para cobrir todo o ciclo de vida do pipeline de dados [22]:

- Reverse-engineering KMs (RKM): Utilizados para extrair metadados físicos de sistemas de dados heterogêneos para dentro do repositório do ODI.[22]
- Journalizing KMs (JKM): Projetados para gerenciar e executar rotinas de Captura de Alterações de Dados (CDC) nos bancos de origem por meio de triggers corporativos ou logs nativos.[22]
- Loading KMs (LKM): Otimizados para extrair as informações brutas das origens e carregá-las de forma acelerada na área de staging de destino.[22, 23]
- Integration KMs (IKM): Responsáveis por consolidar os dados transformados da área de transição nas tabelas finais de destino, implementando estratégias como inserts incrementais ou Dimensões de Variação Lenta (SCD).[22, 24]
- Check KMs (CKM): Utilizados para garantir a qualidade e a consistência dos dados, verificando restrições estruturais de integridade diretamente sobre os fluxos carregados.[22]
- Service KMs (SKM): Projetados para expor transformações e fluxos de dados sob a forma de web services corporativos reutilizáveis.[22]

Talend

Fundado como uma alternativa de código aberto baseada em Java e adquirido pela Qlik em 2023, o Talend Data Fabric diferencia-se das demais plataformas tradicionais por adotar um modelo centrado na geração automática de código nativo.[15, 16, 18] Ao projetar graficamente um fluxo de dados no Talend Studio, o desenvolvedor não gera apenas definições de metadados a serem interpretadas por um motor proprietário em tempo de execução; a plataforma gera fisicamente código-fonte Java legível ou código estruturado para Apache Spark voltado para ambientes de Big Data.[18, 21] Esse código compilado

pode rodar de forma autônoma como um job Java independente em qualquer servidor físico que disponha de uma JVM, oferecendo versatilidade para implantações on-premises, híbridas ou multicloud.[16, 17, 18]

O Talend Studio disponibiliza perspectivas visuais unificadas que facilitam alternar entre diferentes ferramentas de design analítico sem a necessidade de fragmentar o desenvolvimento em múltiplos aplicativos.[18] Outro diferencial do Talend é a facilidade de integração de seus jobs de dados com sistemas de controle de versão padrão de mercado como Git e SVN, além de oferecer suporte nativo para fluxos de CI/CD automatizados.[17]

Para gerenciar a execução em larga escala e otimizar recursos computacionais corporativos, a plataforma oferece uma camada robusta de gerenciamento e paralelização de processamento.[17] Por meio do Talend Administration Center (TAC), os administradores podem definir "servidores virtuais" que agrupam múltiplos servidores físicos de execução de tarefas (*Job Servers*).[17] No momento de disparar uma atividade, o TAC analisa dinamicamente métricas de utilização de CPU, consumo de memória RAM e uso de disco de cada nó para direcionar a execução do job para a máquina física mais disponível do cluster.[17] Adicionalmente, o Talend implementa múltiplas camadas de execução paralela estruturada [17]:

- **Process Flow Parallelization:** Capacidade de orquestrar a execução simultânea de múltiplos jobs ou subjobs a partir do console.[17]
- **Data Flow Parallelization:** Quebra lógica de um lote massivo de registros em pedaços menores para que cada segmento de dados seja processado por threads de execução concorrentes de forma simultânea.[17]
- **tParallelize:** Componente dedicado no Talend Studio que permite ao desenvolvedor definir explicitamente quais caminhos lógicos e fluxos de dados do job devem ser executados em paralelo.[17]
- **Auto/Manual Parallel:** Facilidade de realizar a partição física e coleta de registros por meio dos componentes dedicados tPartitioner, tCollector, tDepartitioner e tRecollector.[17]

Para facilitar o desenvolvimento de mapeamentos de dados complexos, o Talend dispõe do componente integrado tMap, que combina as funções clássicas de expressões lógicas, pesquisas externas (*lookups*), roteamento condicional de dados e junção de origens em um único elemento visual centralizado.[18]

Característica de Design	Informatica PowerCenter	Oracle Data Integrator (ODI)	Talend (Qlik)

Paradigma Computacional	ETL intermediário proprietário baseado em memória [17, 18]	ELT baseado em push-down SQL nativo de banco [20, 21]	Geração de código autônomo em Java ou Apache Spark [18, 21]
Controle de Versão	Sistema proprietário integrado de controle de versão [17]	Armazenamento de revisões lógicas em repositórios relacionais [20]	Integração aberta e nativa com repositórios Git ou SVN [17]
Escalabilidade e Distribuição	Escalonamento vertical de nós proprietários centralizados [17]	Aproveita a capacidade MPP ou de cluster do banco de destino [20]	Mapeamento dinâmico de Job Servers via TAC baseado em hardware [17]
Processamento Paralelo	Paralelização interna gerenciada pelo motor proprietário	Otimização baseada nos planos de execução SQL do RDBMS	Paralelização granular via componentes dedicados (tPartitioner) [17]
Componentes de Mapeamento	Mapeadores visuais individuais conectados via Session [18]	Interfaces lógicas mapeadas em modelos de metadados [20]	Componente unificado tMap para junções e roteamentos [18]
Modelo de Licenciamento	Tradicional corporativo ou consumo variável de IPU's [16]	Atrelado ao licenciamento físico do cluster de banco de dados	Licença anual de subscrição por desenvolvedor Talend Studio [16]

Plataformas de Ingestão e DataOps de Transição: Apache NiFi e StreamSets

À medida que o ecossistema de dados corporativos evoluiu dos tradicionais bancos de dados relacionais e sistemas Hadoop para fluxos volumosos em tempo real originados em dispositivos de Internet das Coisas (IoT), chamadas de API web e barramentos de mensageria complexos, o paradigma clássico de ETL em lotes tornou-se obsoleto. Nesse cenário de transição, ganharam destaque as plataformas

modernas focadas na movimentação contínua de dados com ênfase em práticas de DataOps.[25, 26]

Apache NiFi

O Apache NiFi foi originalmente desenvolvido pela Agência de Segurança Nacional dos Estados Unidos (NSA) para atender a requisitos críticos de movimentação de dados em larga escala global e, posteriormente, doado à Apache Software Foundation em 2014.[26] O design arquitetural do NiFi apoia-se firmemente no conceito de Programação Baseada em Fluxos (FBP), fornecendo um ambiente visual interativo onde os engenheiros projetam pipelines complexos como Grafos Acíclicos Direcionados (DAGs).[25, 26] O elemento central que trafega pelos fluxos do NiFi é o **FlowFile**, uma abstração de dados composta por duas seções lógicas independentes e otimizadas para alto desempenho [26, 27]:

- **Atributos:** Dicionário composto por metadados chave-valor armazenado diretamente na memória RAM ou em um banco de dados leve local (*FlowFile Repository*).[26] Os atributos guiam o roteamento condicional rápido do FlowFile sem a necessidade de ler o corpo físico do arquivo.[26]
- **Conteúdo:** O corpo bruto do dado (que pode ser uma estrutura JSON, um arquivo compactado, um arquivo de log ou dados binários complexos).[26, 28] O conteúdo é mantido de forma segura no *Content Repository*, um repositório baseado em disco altamente otimizado para operações de gravação e leitura paralela sequencial de alto throughput.[26]

Para garantir a durabilidade dos metadados e a recuperação total de desastres após travamentos do servidor, o NiFi utiliza o *FlowFile Repository* para manter o estado atual de trânsito dos registros no cluster.[26] Além disso, a plataforma disponibiliza o *Provenance Repository*, um banco de dados de proveniência de dados imutável que registra cada alteração física, clonagem, roteamento ou descarte de FlowFile ao longo do pipeline analítico.[26] Esse nível extremo de rastreabilidade ponta a ponta fornece auditorias corporativas transparentes para conformidade com normas rígidas como HIPAA, GDPR e SOC2.[26]

O NiFi é operado por meio de uma interface gráfica contendo mais de 300 processadores nativos projetados para conexão com uma gama diversificada de protocolos como SFTP, JDBC, Kafka, HDFS e S3.[26] Para resolver o problema de dados malformados ou divergências de formato, a plataforma utiliza processadores específicos de transformação de registro (como *ConvertRecord*, *UpdateRecord* e *InferAvroSchema*) que se integram de forma direta a um Schema Registry corporativo.[26] Desde a versão 1.5, o NiFi adotou o formato Apache Avro internamente, unificando a experiência de mapeamento de dados independentemente de o formato original ser CSV ou JSON.[28]

A tolerância a falhas e a resiliência no NiFi são asseguradas por mecanismos de tratamento de erro robustos, incluindo o uso de relações de falha explícitas em cada

processador que direcionam FlowFiles rejeitados para filas de mitigação ou tentativas automatizadas via componentes dedicados como `RetryFlowFile`. [26] O gerenciamento de sobrecarga de rede é controlado por políticas de contrapressão (*backpressure*) granulares que bloqueiam temporariamente a ingestão em processadores a montante quando filas downstream atingem limites volumétricos predefinidos. [26]

Para ambientes distribuídos na borda da rede ou dispositivos IoT com restrições severas de hardware, o projeto disponibiliza o **MiNiFi**, uma versão complementar do NiFi ultraleve e sem interface gráfica projetada especificamente para atuar na borda e transmitir dados de forma criptografada para os clusters centrais de processamento usando o protocolo seguro Site-to-Site (S2S). [26]

StreamSets

Lançado em 2014 com o propósito de suprir as lacunas de governança estrutural de pipelines corporativos em produção, o StreamSets foi projetado sob a premissa de que "pipelines devem sobreviver a mudanças". [26] Engenheiros de dados frequentemente enfrentam interrupções severas de carregamento causadas por alterações inesperadas nos esquemas das origens (como a inclusão de uma nova coluna por parte de um microsserviço de vendas). [26] O StreamSets resolve esse gargalo ao implementar uma tecnologia nativa e automática de detecção e adaptação de desvios de esquema (*schema drift*), propagando as alterações de colunas em tempo de execução até o destino analítico de forma dinâmica e sem a necessidade de intervenção humana ou reconfiguração manual do fluxo. [26]

Ao contrário do modelo descentralizado de grafos do NiFi, o StreamSets opera sob um modelo de pipeline estritamente linear e fortemente focado no conceito de registros estruturados. [26, 28] Todo registro de dados ingerido a partir de um conector de entrada (*Origin*) é imediatamente normalizado e convertido para um formato de objeto interno e neutro do StreamSets, garantindo que todos os processadores downstream de transformação operem sobre uma estrutura de dados consistente, independentemente de a fonte primária ser um arquivo de texto, um dump de banco relacional ou uma mensagem de fila. [26, 28]

O pipeline lógico do StreamSets é estruturado por meio de quatro tipos bem delineados de componentes de fluxo [26, 28]:

1. **Origins:** Conectores dedicados à extração e leitura física de fontes de dados externas. [26, 28] Cada pipeline do StreamSets aceita estritamente apenas um único nó de Origin. [28]
2. **Processors:** Blocos computacionais focados nas transformações, filtrações, junções e enriquecimento dos registros normalizados. [26, 28]
3. **Destinations:** Conectores de gravação e persistência final dos dados nos destinos analíticos configurados. [26, 28]

4. **Executors:** Gatilhos focados em ações orientadas a eventos (como disparar uma chamada de API REST ou iniciar um job de compactação externa após a gravação de um lote de arquivos).[26, 28]

Embora o StreamSets apresente uma interface web altamente polida para o gerenciamento de ciclos de vida de pipelines DataOps e suporte a CI/CD corporativo, ele impõe certas limitações de design no dia a dia.[25, 26] A plataforma não exibe filas visuais explicitadas na tela de design computacional.[28]

Adicionalmente, para realizar qualquer alteração simples de parâmetro em um componente, o operador é obrigado a paralisar o pipeline como um todo antes de aplicar a modificação estrutural, reiniciando o processamento do marco inicial após a validação.[26, 28]

A plataforma também carece de mecanismos eficientes de reaproveitamento de configurações de conexão (como parâmetros JDBC), exigindo que o engenheiro preencha repetidamente os detalhes de conexão física em cada processador individual que lê do mesmo banco de dados, além de exigir que todos os componentes estejam devidamente conectados para que a validação de fluxo seja concluída.[28]

A tabela a seguir apresenta os contrapontos técnicos e operacionais mais relevantes entre o Apache NiFi e o StreamSets:

Critério de Comparação	Apache NiFi	StreamSets (IBM)
Padrão de Pipeline	Programação baseada em fluxos (DAG) com filas visuais de trânsito [25, 28]	Modelo linear focado em registros de dados estruturados [26, 28]
Abstração do Dado	FlowFile (Atributos lógicos + Corpo físico sequencial em disco) [26]	Registro normalizado interno e unificado na memória RAM [26, 28]
Controle de Fluxo	Roteamento granular complexo, condicionais rápidos e splits [26]	Pipeline linear estruturado voltado para fluxos uniformes [26]
Tratamento de Mudanças	Exige tratamento e mapeamento de desvios estruturais manuais [26]	Deteção e propagação automatizadas de desvios de esquema (<i>schema drift</i>) [26]

Modificações em Execução	Permite pausar e alterar processadores individuais com fluxo ativo [26, 28]	Exige a parada integral do pipeline de processamento de dados [26, 28]
Mecanismo de Depuração	Auditoria e replay via eventos de proveniência de dados [26]	Console interativo integrado com visualização de registro problemático [26]
Controle de Sobrecarga	Gerenciamento e thresholds de contrapressão ativos nas filas [26]	Gerenciamento operacional integrado no painel do Control Hub [25, 26]
Edge Computing	Disponibiliza o agente MiNiFi para coleta leve na borda de rede [26]	Foco em orquestração centralizada de pipelines corporativos [25, 26]

A Era Moderna do ELT Programático e de Código: Airbyte, Meltano, Stitch e dlt (dltHub)

Com o amadurecimento das práticas de engenharia de software aplicadas à engenharia de dados, o mercado analítico passou a rejeitar interfaces exclusivamente gráficas e de design proprietário.[2, 29] Engenheiros de dados modernos passaram a demandar ferramentas leves, extensíveis e que pudessem ser operadas diretamente como código (*code-first*), facilitando o versionamento através do Git, testes automatizados e implantações orientadas a CI/CD.[29, 30]

Stitch

Lançado originalmente em 2016 como um spin-off da RJMetrics e posteriormente integrado ao portfólio da Qlik via Talend, o Stitch Data consolidou-se como um dos pioneiros no modelo SaaS simplificado de ELT.[31] O principal legado técnico do Stitch foi a criação e disponibilização do **Singer Protocol**, um padrão open-source voltado para estruturar pipelines de dados com alta portabilidade.[31, 32]

O Singer Protocol define uma convenção estrita de comunicação baseada em streams JSON unidirecionais entre dois componentes de software independentes [32]:

- **Taps (Extratores):** Scripts especializados em conectar-se às fontes de origem, extrair os dados lógicos e direcionar mensagens JSON para a saída padrão do console (stdout).[32]
- **Targets (Carregadores):** Scripts encarregados de consumir as mensagens estruturadas enviadas a partir da entrada padrão do console (stdin) e

transcrevê-las de forma física e otimizada no banco de dados analítico ou data lake de destino.[32, 33]

O protocolo especifica estritamente três categorias lógicas de mensagens JSON estruturadas que fluem entre as pontas [32]:

- **SCHEMA:** Mensagem que estabelece a tipagem e a estrutura das colunas dos dados a serem transmitidos de forma declarativa.[32]
- **RECORD:** Mensagem que abriga os dados transacionais de cada linha extraída.[32]
- **STATE:** Mensagem de status que armazena informações críticas para carregamentos incrementais (como o ID do último registro processado ou carimbos de data/hora), permitindo que o pipeline reinicie exatamente do ponto de pausa no ciclo seguinte.[32]

Apesar de o Stitch usufruir de conexões simplificadas com mais de 130 origens nativas, ele apresenta menor profundidade de tratamento para cenários corporativos complexos em comparação com concorrentes focados exclusivamente em grandes volumes, como o Fivetran.[31, 32] Seu modelo de negócios baseia-se em faixas transparentes de contagem de linhas mensais sincronizadas, diferenciando conectores certificados comercialmente daqueles geridos pela comunidade aberta do projeto Singer.[31, 32, 34]

Meltano

O Meltano originou-se como uma iniciativa de engenharia de dados dentro da estrutura de DevOps do GitLab.[31] O framework destaca-se como um orquestrador e gerenciador de pipelines puramente declarativo de linha de comando (CLI-first) e focado em engenharia de software baseada em GitOps.[29, 30] No ecossistema Meltano, os engenheiros não desenham caminhos visuais; eles declaram as especificações de conectores, parametrizações e agendamentos operacionais por meio de arquivos estruturados em formato YAML.[29]

O Meltano apoia-se nativamente no ecossistema de conectores criados sob a especificação do Singer Protocol.[29, 31] Ele fornece uma camada de gerenciamento, instalação automatizada, tratamento de falhas e injeção de segredos ao redor dos *Taps* e *Targets*, permitindo que times que operam com forte rigor de software tratem seus pipelines como códigos-fonte convencionais sujeitos a testes de unidade, linting estrito e automação de deploy contínuo em servidores de orquestração analíticos.[29, 35]

Airbyte

O Airbyte foi concebido para atuar como a principal plataforma de integração de dados moderna de código aberto, apresentando uma biblioteca superior a 600 conectores funcionais e oferecendo interfaces web simplificadas aliadas a uma API declarativa e robusta de nível empresarial.[30, 35] Diferente da simplicidade

puramente baseada em pacotes Python do dlt, o Airbyte OSS é uma plataforma complexa de orquestração distribuída.[29, 35] Em ambientes de teste locais, a ferramenta pode ser executada por meio do utilitário de terminal `abctl`. [36] Para ambientes de produção de larga escala, o Airbyte exige a infraestrutura de orquestração do Kubernetes.[29] Sua execução física consome múltiplos serviços internos integrados de forma persistente, exigindo instâncias ativas do framework Temporal (orquestrador distribuído de tarefas de sync), Redis (cache interno e barramento de sinalização) e PostgreSQL (banco de metadados operacionais), o que eleva substancialmente os custos físicos e a sobrecarga operacional para as equipes de infraestrutura.[29]

O Airbyte disponibiliza o Connector Development Kit (CDK), que simplifica de forma acentuada o processo de desenvolvimento e homologação de conectores personalizados usando código ou interfaces interativas.[35] O conector S3 do Airbyte implementa suporte nativo para mascarar dados sensíveis de PII em tempo de ingestão usando funções criptográficas como SHA-256 e MD5, além de permitir o uso de chaves RSA para decodificação em tempo de gravação no data lake.[37]

dlt (data load tool)

O dlt representa a abordagem de maior flexibilidade e menor overhead de infraestrutura do mercado moderno de movimentação de dados.[29, 30] O dlt é uma biblioteca de código aberto puramente escrita em Python e sem estado computacional físico (*stateless*), que pode ser facilmente inicializada em qualquer ambiente operacional por meio do utilitário de linha de comando `dlt init`. [35, 36]

O dlt foi projetado para rodar de forma distribuída dentro de qualquer rotina ou script de software convencional, facilitando seu deploy integrado em funções serverless de baixo custo (como Google Cloud Functions ou AWS Lambda), contêineres temporários de microsserviços (AWS ECS) ou agendadores de tarefas leves e locais.[29, 36]

A grande inovação técnica introduzida pelo dlt consiste em sua capacidade de realizar a normalização estrutural automática e em tempo real de cargas semiestruturadas de alta complexidade originadas em chamadas de APIs JSON.[29] Ao ingerir dicionários ou arrays de dados com múltiplos níveis de aninhamento de forma dinâmica, o dlt [29, 38]:

1. Analisa e deduz a tipagem de dados das chaves lógicas do JSON recebido.
2. Cria de forma automatizada as tabelas correspondentes no banco de dados ou data lake de destino.
3. Achata os elementos de aninhamento simples e gera tabelas filhas secundárias conectadas de forma relacional às tabelas primárias por chaves estrangeiras exclusivas (`_dlt_id` gerado de forma padronizada pelo dlt).

4. Gerencia de forma transparente a evolução dinâmica de esquemas lógicos à medida que novas colunas ou metadados surgem em transações futuras das APIs de origem.

Em termos de desempenho e eficiência computacional de leitura física, os benchmarks mostram que a utilização de conexões otimizadas em C e Rust por meio da biblioteca dlt associada ao backend de dados ConnectorX e extrações em formato PyArrow entrega velocidades de processamento muito superiores em comparação com o resto do ecossistema de dados modernos.[36, 39] Isso permite quantificar o desempenho computacional de velocidade física aproximada de gravação em destino analítico sob o seguinte comportamento matemático demonstrado pelos benchmarks de engenharia de dados [36, 39]:

Velocidade

dlt + ConnectorX

$\approx 2 \times$ Velocidade

Fivetran

Velocidade

dlt + ConnectorX

$\approx 6 \times$ Velocidade

Airbyte

Velocidade

dlt + ConnectorX

$\approx 10 \times$ Velocidade

Stitch

Isso posiciona a biblioteca dlt como a solução analítica mais rápida e econômica para equipes que priorizam a programação baseada em Python sem custos de manutenção de servidores de virtualização ativos ou infraestruturas complexas de bancos de dados secundários.[29, 36]

Critério de Comparação	Stitch	Airbyte OSS	Meltano	dlt (dltHub)
Padrão de Implantação	Plataforma SaaS gerenciada em nuvem pública [31, 32]	Plataforma web com infraestrutura em contêineres [29, 35]	Ferramenta local baseada em linha de comando (CLI) [29, 30]	Biblioteca leve importada via Python (import dlt) [29, 35]
Protocolo Adotado	Singer Protocol (taps e targets oficiais e comunitários) [32]	Protocolo de conectores nativo do ecossistema Airbyte [30, 35]	Singer Protocol como base de dados estruturada [29, 30, 31]	Dialeto de especificação Python nativo com suporte a Arrow [29, 40]
Gargalo de Infraestrutura	Dependência de cotas de rede SaaS e taxas de latência do provedor	Elevado consumo de RAM/CPU de nós Kubernetes e Temporal [29]	Dependência de servidores e orquestradores de deploy de terceiros [29]	Totalmente dependente do ambiente físico onde o script roda [29, 36]
Evolução do Esquema	Evolução de esquema básica estruturada pelo target	Gerenciamento e mapeamento declarativo na interface web [37, 41]	Gerenciado de forma transparente pelo target de banco de dados	Criação dinâmica de tabelas relacionais e achatamente [29, 38]
Controle de PII	Mascaramento básico configurado no plano premium	Criptografia integrada via RSA e funções SHA de terminal [37]	Customizável por meio de plugins de interceptação lógicos	Exige manipulação prévia do dado no script de execução Python

Integração de Baixa Latência com Captura de Alterações de Dados (CDC): AWS DMS e Debezium

Para atender à necessidade de processar informações em tempo real e alimentar motores de análise preditiva com dados de alta frescura (*freshness*), as técnicas tradicionais de polling por consultas SQL relacionais periódicas provaram-se ineficientes.[42] O polling excessivo onera os servidores de bancos de dados transacionais com varreduras constantes e adiciona latências operacionais indesejadas.[42] A Captura de Alterações de Dados (CDC) supera essas restrições ao interceptar as alterações diretamente nos logs de transações imutáveis gravados pelos bancos de dados no disco rígido físico antes de sua gravação definitiva.[42, 43]

Debezium

O Debezium destaca-se como o framework de CDC de código aberto dominante no mercado analítico moderno, construído para atuar de forma nativa e integrada com o ecossistema do Apache Kafka.[43] O Debezium é implantado como um conjunto de conectores executados sobre os nós de processamento distribuído do Kafka Connect.[43] Ele funciona por meio da conexão física e monitoramento contínuo dos logs transacionais privados de motores de bancos de dados como PostgreSQL, MySQL, SQL Server e Oracle.[43, 44] No PostgreSQL, por exemplo, o banco de dados de origem precisa ter a propriedade de replicação lógica ativada (`rds.logical_replication = 1`), permitindo que os conectores do Debezium monitorem os registros contidos no Write-Ahead Log (WAL) de forma transparente sem onerar a concorrência de leituras das tabelas operacionais.[44]

Ao capturar uma alteração de nível de linha em tempo de execução, o Debezium empacota e publica essa alteração sob o formato de um evento JSON ou Avro contendo um envelope de metadados robusto e detalhado [43, 45]:

- **Before:** Seção estruturada que reflete as chaves e os valores lógicos de todas as colunas da linha afetada antes de a transação de alteração ser executada no banco de dados.[43]
- **After:** Seção estruturada que reflete as chaves e os valores lógicos de todas as colunas após o commit da transação de escrita.[43]
- **Source:** Detalha informações de auditoria interna da origem, como o nome da base física de dados, tabela alterada, ID do processo transacional, carimbo de data/hora e a posição física do log.[43, 45]
- **Op:** Enumera o tipo exato de operação transacional detectada pelo log (c para criação/insert, u para atualização/update, d para exclusão física/delete e r para leituras de snapshot de sincronização inicial).[43, 45]
- **Ts_ms:** Fornece um carimbo de data/hora unificado em milissegundos para registrar o momento exato em que a mensagem de CDC foi capturada e serializada.[43]

Esse stream persistente de mensagens ricas é publicado em tópicos unificados do Kafka com latência na casa dos milissegundos, permitindo que dezenas de sistemas analíticos downstream consumam a mesma fonte de alterações de forma independente e sem causar gargalos adicionais na origem.[43, 46]

Para implantar essa arquitetura em escala em nuvens gerenciadas, é comum associar o Debezium a recursos serverless da AWS.[44] Em uma infraestrutura típica de produção para Amazon RDS PostgreSQL, o conector do Debezium Connect é baixado em formato compactado (`tar.gz`) dentro de uma instância EC2, zípadado na estrutura exigida pelo AWS MSK Connect e enviado via CLI para um bucket do Amazon S3.[44] Em seguida, o serviço gerenciado Amazon MSK Serverless Connect cria e escala de forma automatizada os trabalhadores necessários para executar o conector e transmitir os eventos de alteração diretamente do banco de dados para os tópicos do Kafka Serverless.[44]

No entanto, as equipes de engenharia devem monitorar de forma estrita a métrica de armazenamento de disco de replicação do banco (`ReplicationSlotDiskUsage` no CloudWatch), uma vez que, caso os consumidores ou os agentes do Debezium fiquem paralisados por períodos de manutenção prolongados, os segmentos físicos de WAL no PostgreSQL continuam a se acumular rapidamente no disco de produção do banco transacional, podendo esgotar o armazenamento físico e travar as operações transacionais críticas do negócio.[45]

AWS Database Migration Service (DMS)

O AWS Database Migration Service (DMS) é um serviço de nuvem proprietário e totalmente gerenciado cuja principal missão é facilitar a movimentação física acelerada e segura de bancos de dados transacionais para o ambiente de nuvem da AWS.[43] O uso do CDC contínuo no DMS funciona como uma funcionalidade secundária para garantir que os alvos analíticos permaneçam consistentes durante a janela de replicação final antes de um corte definitivo de sistemas, ou para a sincronização básica de data lakes e bases secundárias.[43]

O funcionamento operacional do DMS apoia-se em instâncias de replicação provisionadas e configuradas por tarefas lógicas.[43] O operador define endpoints lógicos de entrada (origem) e de saída (destino) e cria regras de seleção de dados estruturados.[43] O DMS encarrega-se de gerenciar a leitura dos logs nativos e executar a persistência diretamente no banco de destino ou em arquivos brutos delimitados no Amazon S3.[43]

Entretanto, o DMS apresenta menor flexibilidade e semântica de streaming que o Debezium.[43] Ele funciona prioritariamente como um sistema de replicação ponta a ponta focado no estado final dos registros estruturados, aplicando as alterações linha por linha diretamente no destino, sem disponibilizar nativamente um histórico distribuído, imutável e reexecutável de eventos lógicos de transação que possa ser livremente acessado por outros microsserviços.[43] Caso o usuário deseje extrair

uma semântica de streaming rica a partir do DMS, ele precisa configurar arquiteturas adicionais de alta complexidade, direcionando as gravações do DMS para fluxos do Amazon Kinesis ou Amazon MSK.[43]

Critério Arquitetural	Debezium Connect	AWS Database Migration Service (DMS)
Abordagem de Sistema	Plataforma de CDC open-source nativa de streaming distribuído [43]	Serviço em nuvem totalmente gerenciado focado em migração [43]
Arquitetura de Deploy	Executado de forma distribuída no cluster Kafka Connect [43]	Instâncias de replicação provisionadas elasticamente na AWS [43]
Formato das Alterações	Registros contendo envelopes ricos de dados estruturados [43, 45]	Gravação estruturada das alterações diretamente no destino final [43]
Latência de Processamento	Sub-segundo consistente devido ao streaming em memória [43, 46]	Baixa latência, sujeita a limites e processamento em lote de tarefas
Reconsumo de Logs	Altíssimo (os tópicos retêm os eventos transacionais no Kafka) [43]	Baixo (aplica as alterações no destino de forma ponta a ponta) [43]
Dependência de Cloud	Agnóstico (roda em ambientes on-premises, híbridos ou nuvem) [43]	Fortemente integrado e acoplado ao ecossistema de nuvem AWS [43]
Roteamento de Tabelas	Avançado via transformações de mensagem única (SMTs) [45]	Regras de mapeamento e filtros lógicos de tarefa simplificados [43]

Ingestão Moderna para Formatos de Tabela Abertos: Integrando Airbyte, dlt e CDC com Iceberg, Delta Lake e Hudi

Com a consolidação de infraestruturas modernas de Data Lakehouse, o maior desafio dos processos de engenharia de dados consiste em garantir transacionalidade ACID, evolução estruturada de esquemas e controle de atualizações lógicas em tempo de execução diretamente sobre repositórios de armazenamento de objetos em nuvem.[47, 48] Para atingir essa finalidade, as ferramentas modernas de ingestão precisam se integrar de forma íntima com os formatos de tabela abertos: Apache Iceberg, Delta Lake e Apache Hudi.[47, 48, 49]

Apache Iceberg e as Soluções de Gravação e Consumo

O Apache Iceberg adota uma especificação de metadados focada na neutralidade de motores analíticos, organizando a tabela em uma estrutura hierárquica baseada em arquivos JSON de definição de estado de snapshot, arquivos de lista de manifestos (*manifest lists*) e arquivos de manifesto (*manifest files*) que mapeiam diretamente os arquivos de dados Parquet físicos armazenados na nuvem.[48]

O conector do Airbyte para destino do tipo S3 Data Lake implementa suporte robusto e nativo para o formato Iceberg.[41] Para resolver a concorrência e evitar a duplicação física de registros, o Airbyte adota uma estratégia de Mesclagem na Leitura (*Merge-on-Read* - MOR), traduzindo as chaves primárias dos streams originais em colunas de identificação lógicas do Iceberg.[41] Os dados novos ou modificados são persistidos ao lado de arquivos de exclusão estruturados baseados em igualdade.[41]

Nas sincronizações de truncamento analítico total (*truncate syncs*), o Airbyte implementa uma estratégia de ramificação atômica de carregamento para evitar leituras de dados parciais ou inconsistências no data lakehouse.[37] Os novos dados são inicialmente gravados em uma ramificação de metadados temporária e isolada chamada `airbyte_staging`; após a conclusão e validação bem-sucedida de todas as transações do lote de ingestão, o conector dispara um comando de avanço rápido (*fast-forward*) atômico na ramificação principal (`main`), atualizando o catálogo ativo do Iceberg de forma transparente para as ferramentas de visualização downstream.[37]

Para cenários baseados em código Python puro, o dlt implementa suporte acelerado para gravação de tabelas Iceberg por meio da biblioteca `pyiceberg`. [38] O dlt automatiza a criação estrutural e a evolução contínua das tabelas no destino analítico por meio de catálogos baseados em SQL (comumente utilizados para fins de depuração ou desenvolvimento local) e catálogos baseados em REST focados em ambientes corporativos de alta escala (como Polaris e Lakekeeper).[38, 50]

O dlt utiliza uma integração baseada em classes integradas ao seu cliente de carga de trabalho (`complete_load`): os arquivos Parquet estruturados durante a

extração são registrados em tempo de execução e, após a conclusão total do lote de dados, o dlt realiza o commit unificado de todos os novos caminhos de arquivos em um único snapshot transacional do Iceberg, garantindo transações atômicas consistentes.[50] O dlt também disponibiliza funções auxiliares de adaptação de partição por meio do método `iceberg_adapter`, permitindo particionar as tabelas fisicamente por partições de identidade lógica ou por transformações temporais e estruturais (por ano, mês, dia, hora, buckets de hash ou limitação de tamanho de strings).[38, 50]

Delta Lake e a Integração com Rust e Nuvem

O Delta Lake gerencia sua transacionalidade analítica por meio do registro sequencial de transações armazenado na pasta estrutural física `_delta_log` presente no diretório de cada tabela analítica.[40, 51] O dlt fornece suporte nativo para a escrita de tabelas Delta de forma direta por meio de sua integração baseada no sistema de arquivos do pipeline.[40, 52] O dlt consome os arquivos Parquet higienizados gerados na etapa de normalização, monta estruturas em formato Apache Arrow e as injeta diretamente no conector de baixo nível escrito em linguagem Rust `delta_lake`. [40]

Entretanto, as equipes de desenvolvimento devem observar uma restrição conhecida da biblioteca Rust em produção: o processamento físico de volumes massivos de dados voltados para uma mesma tabela Delta analítica gera picos severos de consumo de memória RAM do sistema.[52] Para mitigar esses travamentos de OOM, recomenda-se segmentar as execuções do pipeline do dlt em lotes menores e incrementais.[52]

Para expandir o acesso analítico sem duplicar o armazenamento físico dos dados na nuvem, o Delta Lake disponibiliza a tecnologia **Delta Lake UniForm**. [53] O UniForm gera automaticamente metadados lógicos compatíveis com a especificação do Apache Iceberg em tempo de execução ao mesmo tempo em que escreve o log transacional nativo do Delta Lake.[53] Uma vez que os dois arquivos de metadados lógicos apontam estritamente para o mesmo conjunto físico de arquivos de dados Parquet subjacentes, motores analíticos nativos de Iceberg (como Amazon Redshift, Athena ou Snowflake) podem ler os dados de forma transparente sem a necessidade de realizar conversões computacionais de dados.[37, 53]

CDC Incremental e Estratégias de Compactação nos Formatos de Tabela

As particularidades e restrições estruturais que guiam a ingestão contínua de CDC e a compactação física de arquivos pequenos diferenciam de forma evidente os três principais formatos de tabela abertos em data lakehouses.[49]

No **Apache Hudi**, a arquitetura de gravação foi concebida sob o viés do suporte nativo e otimizado a processos de streaming contínuos e upserts pesados originados em CDC.[49] O utilitário nativo *DeltaStreamer* permite criar pipelines

incrementais robustos com baixo esforço de codificação, processando diretamente arquivos de log em formato de gravação rápida baseada em linhas (como arquivos Avro em tabelas Merge-on-Read).[49]

O Hudi lida de forma ativa com o problema de arquivos pequenos em tempo de execução ao gerenciar o tamanho das escritas Parquet no momento de gravação ou ao disparar rotinas de compactação e agrupamento (*clustering*) física de forma assíncrona, minimizando a perda de performance de leitura analítica sem concorrer ou travar o pipeline de escrita ativo.[49] Os deletes para conformidade de privacidade (como requisições LGPD) são geridos de forma automática pelo serviço de limpeza interno (*Cleaner*) do Hudi.[49]

No **Apache Iceberg**, o processamento incremental de dados baseia-se prioritariamente no uso de instruções lógicas do tipo MERGE INTO.[49] Essa instrução executa uma operação complexa de junção (*join*) física de dados que consome recursos computacionais expressivos, demandando varreduras completas em tabelas e partições analíticas de grande porte caso os dados não estejam previamente indexados ou ordenados por colunas de chaves.[49]

O Iceberg não executa serviços automatizados ou integrados de otimização de arquivos ou compactação durante a escrita de novos dados.[49] Sendo assim, pipelines que gravam fluxo contínuo ou CDC no Iceberg criam progressivamente gargalos de arquivos fragmentados, exigindo que o engenheiro programe e execute jobs de compactação e reordenação física periódicos via motores externos como Apache Spark de forma desacoplada para evitar a degradação estrutural do data lakehouse.[49]

No **Delta Lake**, o controle de capturas lógicas incrementais e atualizações de CDC conta com o recurso nativo **Change Data Feed (CDF)**. [49] Quando o CDF está ativo em uma tabela Delta, a plataforma grava logs adicionais de alterações estruturais diretamente ao lado dos arquivos transacionais, permitindo extrair e consumir apenas as modificações estruturais geradas em snapshots específicos.[49]

Apesar desse recurso, o Delta Lake também não consolida ou compacta arquivos Parquet de forma automatizada no ato de gravação do pipeline de ingestão.[49] O gerenciamento estrutural de arquivos pequenos e a limpeza de versões históricas lógicas de tabelas expiradas exigem que o operador dispare de forma controlada comandos de manutenção manuais ou parametrizados (como as instruções OPTIMIZE e VACUUM) para reconstruir os índices transacionais e compactar o armazenamento do data lakehouse de forma consistente.[49]

A tabela a seguir apresenta os detalhes operacionais de manipulação de streaming, CDC e compactação nos três formatos de tabela aberta:

Característica de Engenharia	Apache Iceberg	Delta Lake	Apache Hudi
Ingestão Streaming	Rápida para fluxos puramente de anexação (<i>append-only</i>) [49]	Rápida para fluxos puramente de anexação (<i>append-only</i>) [49]	Otimizada nativamente para fluxos de upsert e escritas incrementais [49]
Captura de CDC Nativa	Exige criação e manutenção de visões de log de alterações (<i>changelog views</i>) [49]	Suporte estruturado por meio do recurso Change Data Feed (CDF) [49]	Suporte nativo por consultas de leitura incremental integradas [49]
Formato de Payload CDC	Sem suporte nativo a payloads brutos (exige Flink CDC ou customização) [49]	Dependente do motor Spark ou lógica declarada na gravação analítica	Suporte nativo de ingestão a formatos brutos como Debezium e DMS [49]
Compactação Física	Executada exclusivamente por meio de processos externos assíncronos [49]	Executada de forma manual ou agendada via instrução OPTIMIZE [49]	Gerenciada de forma assíncrona por serviços integrados [49]
Limpeza de Histórico	Exige execução periódica de rotinas de manutenção customizadas [49]	Gerenciada por meio de rotinas periódicas de instrução VACUUM [49]	Gerenciada de forma automática pelo serviço nativo do Hudi <i>Cleaner</i> [49]

-
1. ETL vs ELT: Differences, Use Cases, Costs, Tradeoffs - Domo, <https://www.domo.com/glossary/etl-vs-elt>

2. Major data movement patterns: ETL, ELT, CDC & more - dbt Labs,
<https://www.getdbt.com/blog/data-movement-patterns>
3. ETL vs. ELT—Key Differences, Improvements, and Trends | Nexla,
<https://nexla.com/data-integration-techniques/etl-vs-elt/>
4. ETL vs. ELT: The Evolution of Data Integration - DEV Community,
https://dev.to/de_clerke/etl-vs-elt-the-evolution-of-data-integration-1ep
5. What is Hadoop? | Databricks,
<https://www.databricks.com/blog/what-is-hadoop>
6. Differences between Sqoop and Flume - DataCouch,
<https://datacouch.io/blog/differences-between-sqoop-and-flume/>
7. A Comprehensive Guide On Apache Sqoop - Analytics Vidhya,
<https://www.analyticsvidhya.com/blog/2022/09/a-comprehensive-guide-on-apache-sqoop/>
8. Apache Sqoop moved into the Attic in 2021-06 - Stack Overflow,
<https://stackoverflow.com/questions/71108443/apache-sqoop-moved-into-the-attic-in-2021-06>
9. Board Meeting Minutes - Sqoop - Apache Whimsy,
<https://whimsy.apache.org/board/minutes/Sqoop.html>
10. Apache Sqoop projects retirement : r/dataengineering - Reddit,
https://www.reddit.com/r/dataengineering/comments/ssk357/apache_sqoop_projects_retirement/
11. What is Apache Flume? - Dremio, <https://www.dremio.com/wiki/apache-flume/>
12. Apache Flume and Logging for Data Pipelines | Logz.io,
<https://logz.io/blog/apache-flume-and-data-pipelines/>
13. Apache Flume: Introduction to Data Extraction and Movement - Confluent,
<https://www.confluent.io/learn/apache-flume/>
14. Apache Flume Features & Limitations of Apache Flume - DataFlair,
<https://data-flair.training/blogs/flume-features-limitations/>
15. Informatica vs Talend vs Integrate.io,
<https://www.integrate.io/blog/informatica-vs-talend-vs-integrateio/>
16. Informatica vs Talend vs CloverDX: Compare top data integration platforms (2026),
<https://www.cloverdx.com/blog/informatica-vs-talend-vs-cloverdx-compare-top-data-integration-platforms-2026>
17. Talend vs. Informatica PowerCenter: What's the Difference? - Part 3,
<https://www.talend.com/resources/informatica-powercenter-developers-guide-to-talend-part-3/>
18. Talend vs. Informatica PowerCenter: What's the Difference?,
<https://www.talend.com/resources/informatica-powercenter-developers-guide-to-talend-part-1/>
19. Qlik Talend Cloud vs Informatica | 2026 Data Integration & ETL Comparison,
<https://www.qlik.com/us/informatica-migration>

20. Why ODI? A Look Into Oracle Data Integrator Architecture - KPI Partners, <https://www.kpipartners.com/blogs/bid/148184/why-odi-a-look-into-oracle-data-integrator-architecture>
21. Oracle Data Integrator Enterprise Edition, <https://www.oracle.com/integration/data-integrator-enterprise-edition/>
22. 1 Introduction to Oracle Data Integrator, <https://docs.oracle.com/en/middleware/fusion-middleware/data-integrator/14.1.2/quick-ref/introduction.html>
23. A Complete Guide to Oracle Data Integrator Architecture and Components - Tech Burner, <https://itechburner.home.blog/2025/07/10/oracle-data-integrator-architecture-and-components/>
24. Developing a Knowledge Module in Oracle Data Integrator, <https://www.oracle.com/technical-resources/articles/bethke-odi.html>
25. StreamSets vs Apache NiFi | What are the differences? - StackShare, <https://stackshare.io/stackups/apache-nifi-vs-streamsets>
26. Apache NiFi vs StreamSets: Best Data Pipeline Tool? - Ksolves, <https://www.ksolves.com/blog/big-data/nifi/compare-with-streamsets>
27. Compare Apache NiFi vs. IBM StreamSets - G2, <https://www.g2.com/compare/apache-nifi-vs-ibm-streamsets>
28. Open Source ETL: Apache NiFi vs Streamsets - Cube Blog, <https://cube.dev/blog/open-source-etl>
29. Open Source Series: Meltano vs Airbyte vs dlt - Leolytix, <https://www.leolytixco.com/blog/open-source-series-meltano-vs-airbyte-vs-dlt>
30. Top 15 Best ETL Tools 2026: Compare ETL & ELT Platforms (Connectors, Pricing, Deployment) | Weld Blog, <https://weld.app/blog/best-etl-tools>
31. Stitch | Data Ecosystem Wiki | TextQL, <https://textql.com/wiki/stitch>
32. Stitch Review 2025 - Features, Pricing & Alternatives | Workflow Automation, <https://workflowautomation.net/reviews/stitch>
33. Send data to the Singer Stitch target, <https://www.singer.io/target/stitch/>
34. Stitch SaaS Integrations | Stitch Documentation - Stitch Data, <https://www.stitchdata.com/docs/integrations/saas>
35. Airbyte vs dltHub, <https://airbyte.com/compare/airbyte-vs-dlthub>
36. Self hosted tools Benchmarking - dltHub, <https://dlthub.com/blog/self-hosted-tools-benchmarking>
37. Build an AI Application with Data at Scale with the Airbyte S3 Data Lake connector built on Apache Iceberg, <https://airbyte.com/blog/build-once-and-query-anywhere-with-airbytes-data-lake-connector>
38. Destination: Iceberg | dlt Docs - dltHub, <https://dlthub.com/docs/hub/ingestion/iceberg>
39. SQL Benchmarking: comparing data pipeline tools - dltHub, <https://dlthub.com/blog/sql-benchmark-saas>

40. Delta Lake | dlt Docs - dltHub,
<https://dlthub.com/docs/dlt-ecosystem/destinations/delta-iceberg>
41. S3 Data Lake Connector | Airbyte Documentation,
<https://docs.airbyte.com/integrations/destinations/s3-data-lake>
42. What is Change Data Capture? - Databricks,
<https://www.databricks.com/blog/what-is-change-data-capture>
43. Debezium vs AWS DMS: Change Data Capture Comparison ...,
<https://risingwave.com/blog/debezium-vs-aws-dms-cdc-comparison/>
44. Implementing real-time change data capture with Debezium for Amazon Aurora PostgreSQL and Amazon RDS for PostgreSQL | AWS Database Blog,
<https://aws.amazon.com/blogs/database/implementing-real-time-change-data-capture-with-debezium-for-amazon-aurora-postgresql-and-amazon-rds-for-postgresql/>
45. Real-time CDC from Aurora PostgreSQL to Amazon S3 Tables using Debezium and Firehose | AWS Big Data Blog,
<https://aws.amazon.com/blogs/big-data/real-time-cdc-from-aurora-postgresql-to-amazon-s3-tables-using-debezium-and-firehose/>
46. Best CDC Tools Compared: A 2026 Guide to Change Data Capture Platforms - Streamkap,
<https://streamkap.com/resources-and-guides/best-cdc-tools-compared>
47. Modern Table Formats for Data Lakehouse Architectures: A Comprehensive Analysis of Apache Iceberg, Delta Lake, and Apache Hudi | International Journal of Computational and Experimental Science and Engineering - ijcesen, <https://www.ijcesen.com/index.php/ijcesen/article/view/4867>
48. Comparison of Data Lake Table Formats (Apache Iceberg, Apache Hudi and Delta Lake),
<https://www.dremio.com/blog/comparison-of-data-lake-table-formats-apache-iceberg-apache-hudi-and-delta-lake/>
49. Choosing an open table format for your transactional data lake on ...,
<https://aws.amazon.com/blogs/big-data/choosing-an-open-table-format-for-your-transactional-data-lake-on-aws/>
50. sidequery/dlt-iceberg: An Iceberg destination for DLT that supports REST catalogs - GitHub, <https://github.com/sidequery/dlt-iceberg>
51. Load Data into Delta Lake on Databricks Lakehouse - Airbyte,
<https://airbyte.com/tutorials/load-data-into-delta-lake-on-databricks-lakehouse>
52. Destination: Delta | dlt Docs - dltHub,
<https://dlthub.com/docs/hub/ingestion/delta>
53. Expand data access through Apache Iceberg using Delta Lake UniForm on AWS,
<https://aws.amazon.com/blogs/big-data/expand-data-access-through-apache-iceberg-using-delta-lake-uniform-on-aws/>